Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

PHP

PHP

- Stands for:
  - Past: Personal Home Page
  - Now: PHP Hypertext Preprocessor
- Provides the procedural and object-oriented programming at the web server (*server-side scripting*)
- PHP code is compiled and processed by web server, then the result will be showed by browser (unlike HTML, CSS, or JS, PHP code can’t be seen using browser)

Deploy web application in XAMPP

- Run the Apache web server
- Create a folder, i.e. “hellophp” inside this path: “../xampp/htdocs/”
- Create a .php file inside “hellophp” folder

```
<?php
  echo "hello PHP!";
?>
```

- Call that file using browser with address below:
  - localhost/hellophp
  - or, 127.0.0.1/hellophp

Code commenting

- Same with C++ / Java / Javascript
  - `// one line`
  - `/* ..`

    `multi-line`

    `.. */`

Variables & Data Types

- Rules of naming
  - Case sensitive
  - Start with a letter or underscore
  - Must not a PHP's reserved word
  - Must not contains operational characters
- Variable declaration

```
$namevariable = value;
```

- Data types:
  - Standard (boolean, int, float/double, string)
  - Array
  - NULL
  - Resource (pre-defined variables)

Example: standard data types

- `$flag = true;` (boolean)
- `$i = 2;` (integer)
- `$j = 2.3;` (float / double)
- `$name1 = "double";` (string double quote)
- `$name2 = 'single';` (string single quote)

Example: array data type

- Initialization

```
//numeric index
$arr1 = array();
$arr2 = array(1303, "IT");
//associative index (map)
$arr3 = array("code"=>1303,
                "name"=>"IT");
```

- Call

```
echo $arr2[1]; //output: IT
echo $arr3["kode"]; //output: 1303
```

Example: array data type

- Assignment

```
$arr2[1] = "S2 IF";
$arr3["code"] = 2301;
```

NULL data type

- NULL is different from 0

```
$namavar = NULL;
//to check a variable isn't NULL, use isset()
if (isset($namavar)) { //false
…
}
$namavar = 5;
if (isset($namavar)) { //true
…
}
```

Resource data type

- Special variable for PHP information and specific operation

- Example:
  - `$_SERVER["HTTP_HOST"];`
  - `$_SESSION["user"];`
  - `$_GET["id"];`

- Session and Get will be discussed later

Operator

- Arithmetic: + - * / % ++ --
- Assignment : = += -= *= /= %=
- Comparison : == != > < >= <=
- Logical: && || ! AND OR XOR
- String concat: . .=

Conditional (if)

- Example:

```
<?php
 $i = 1;
 if ($i>0) {
    echo $i." = positive number <br />";
 }
?>
```

Conditional (if-else)

- Example:

```
<?php
 $i = -8;
 if ($i>0) {
    echo "positive";
 } else if ($i==0) {
  echo "zero";
 } else {
  echo "negative";
 }
?>
```

Conditional (switch-case)

- Example:

```
<?php
 $bil=5;
 switch ($bil) {
   case 0 :
      echo "zero";
      break;
   case 2 :
      echo "two";
      break;
   default :
      echo "other";
 }
?>
```

Looping (while)

- Example:

```
<?php
 $i=0;
 while ($i<4) {
   echo $i. "<br />";
   $i++;
 }
?>
```

Looping (do-while)

- Example:

```
<?php
  $i=0;
 do {
   echo $i. "<br />";
   $i++;
 } while ($i<8);
?>
```

Looping (for)

- Example:

```
<?php
  for ($i=1;$i<7;$i++) {
   echo '<h'.$i.'>H'.$i.'</h'.$i.'>';
   //echo  "<h$i>H$i</h$i>";
  }
?>
```

Looping (foreach)

- Example:

```
<?php
    $arr = array("IT","SE","IF");
   foreach ($arr as $x) {
      echo "prodi ".$x."<br />";
   }
?>
```

Function

- PHP function is similar to Javascript function:
  - Begins with keyword ‘function’, not data type or void
  - Can have a return value or not

Function

- Example:

```
<?php
 //function declaration
   function sayhello($str) {
      echo "hello ".$str;
   }
   function add($a,$b) {
      return $a+$b;
  }
 //function execution
   sayhello("bro! <br />");
   $x = add(5,6);
   echo $x."+ 3 = ".add($x,3);
?>
```

Form Data Submitting

- GET method
  - After form is submitted, form data is appended in URL
  - How to get the submitted data: `$_GET["nama_name"]`
  - nama_name is the value of your "name" attribute in HTML tag

- POST method
  - After form is submitted, data is stored as resource, not seen in URL
  - How to get the submitted data: `$_POST["nama_name"]`
  - nama_name is the value of your "name" attribute in HTML tag

When to use GET or POST

- If data is confidential or for manipulation, use POST (example: login, registration, data input, data update)
- If data submitted is large, use POST, so the URL doesn't become too long (example: lots of fields/inputs, has attachment)
- Other than above (data will be used generally), use GET (example: searching, page/hyperlink, pagination)

Example: GET

- find.php

```
<?php
 echo "Result = ".$_GET["key"];
?>
```

- formfind.php

```
<form method="get" action="find.php">
 Keyword : <input type="text" name="key" /><br />
 <input type="submit" value="Search" />
</form>
```

Example: POST

- process.php

```
<?php
echo "User = ".$_POST["user"].",<br /> Pass = ".$_POST["pass"];
?>
```

- formlogin.php

```
<form method="post" action="process.php">
 Username: <input type="text" name="user" /><br />
 Password: <input type="password" name="pass" /><br />
 <input type="submit" value="Login" />
</form>
```

Session & Cookie

- Resource variable to store data globally
- Usually used for storing confidential or repetitive information (example: login status, account info, e-commerce cart, etc)

Session

- Data (variable) is stored and managed by web server
- Time-out duration can be configured in the code, or taken from web server configuration
- Every client (browser) will be given different session id
- When client is closed, data is automatically erased

Session

- Starting the session (usually put at the top of your code)

```
<?php
  session_start();
  echo "Session ID Anda : ".session_id();
  //... kode selanjutnya
?>
```

Session

- Storing & accessing session

```
<?php
    session_start();
    if (!isset($_SESSION["counter"])) {
        $_SESSION["nama"] = "prakwebprosession";
        $_SESSION["counter"] = 0;
    }
    echo "<br /> Nama : ".$_SESSION["nama"];
    $_SESSION["counter"]++;
    echo "<br /> Anda telah melihat halaman ini sebanyak : ".
          $_SESSION["counter"]." kali";

?>
```

Session

- Deleting session

```
<?php
  session_start();
  unset($_SESSION["nama"]); //delete specific session data
  session_destroy(); //delete all session data
?>
```

Cookie

- Data (variable) is stored and managed by client (browser)
- Expire time can be configured in the code, or taken from browser configuration. As long as data has not expired or deleted manually (by clearing browser data etc), data can be used over again even the browser is closed
- Data can be accessed at the next request
- Function: setcookie(varname, value, expire, path, domain, is_secure)

Cookie

- Varname: Cookie variable name
- Value: The value / data we want to store
- Expire: Cookie expire time. For example, if you fill it with `time()+3600` then it will expire after 1 hour. If expire time is not set then the cookie will be expired when the browser is closed (like session)

Cookie

- Storing cookie

```
<?php
  setcookie("nama","prakwebprocookie");
  setcookie("nama2","cookie exp",time()+30);
?>
```

Cookie

- Accessing cookie

```
<?php
  echo "<br /> Nama kue : ".$_COOKIE["nama"];
  echo "<br /> Nama kue : ".$_COOKIE["nama2"];
?>
```

Cookie

- Deleting cookie

```
<?php
  setcookie("nama","");
  setcookie("nama2","",time()-30);
?>
```

Any question?

References

- https://www.w3schools.com/php/default.asp

Fakultas Informatika
School of Computing
Telkom University

THANK YOU